

什么是 AI Agent?

一句话区分：LLM 负责“推理和生成”，Agent 系统负责“把模型放进可行动、可观察、可评测的环境里”

AI Agent 是一个能在给定目标和约束下，观察环境、选择动作、调用工具、接收反馈，并持续推进任务的软件系统。

Agent ≠ Chatbot

类型	主要能力	是否行动	典型例子
Chatbot	对话和回答	否	FAQ、知识问答
Workflow	按固定步骤处理任务	有，但流程固定	分类 -> 检索 -> 生成 -> 审核
Agent	模型根据状态选择下一步动作	有，路径可变	研究助手、Web Agent、Coding Agent
Multi-Agent	多个 Agent 通过角色或协议协作	有	planner + executor + reviewer

agent最小闭环

常见的 ReAct 思路：Reasoning + Acting；

Goal

-> Observe state 观察状态

-> Think / plan 思考，规划

-> Act with tools 执行工具

-> Observe result 观察结果

-> Stop / continue / ask human 停止/继续/人工

agent系统中必要考虑的几个部分

- 目标：定义任务和成功标准→怎么判断任务完成？
- 策略：系统规则、安全边界、权限→哪些动作需人工确认？
- 状态：当前进度、历史、临时产物→长任务如何恢复？
- 记忆：可复用经验和用户偏好→什么值得存，什么时候忘？
- 上下文构建：组装模型输入→如何避免上下文污染？
- 工具注册表：声明可以调用的工具→schema、错误、权限如何设计？
- 循环控制：决定下一步和停止条件→如何防止死循环？
- 评估与追踪：记录和评估行为→如何证明agent有效？

目标是否可验证？不要只说'帮我优化'-->'让测试通过，减少重复代码，保持接口不变等'

权限是否分级？读、写、删、执行、联网、发消息等，要分开可控制

工具失败怎么办？重试、换工具、降级、问用户

成本是否可控？循环次数、token、工具调用次数限制

做一个agent项目前要回答以上问题，**并确定什么时候该不该用agent，不是每走一步都设一个agent**；比如：任务流程固定→workflow更稳 工具风险高，没权限和回滚机制等 简单文本生成，不需环境反馈

agent工作模式

模式一： ReAct(Reasoning+Acting)边想边做

思考→行动→观察→思考→...→...，如此循环，直至任务完成。适合不确定性强类的任务

eg: 我现在要收拾行李准备去北京出差

thought: 去北京出差3天，先看看天气怎么样 action: 打开手机看天气和温度 observation: 下雨天气，气温低 thought: 应该准备伞和外套 action: 找出来放行李箱中 thought: 出差3天，住哪 action: 打开手机找酒店
.....

优点：透明，可审查，每个环节可见，灵活通用性强

缺点：token消耗大，每一步都需要完整推理，容易死循环

解决方法：最大循环次数，工具调用次数，失败重试上限，每轮记录当前结论，不确定时间问人工

模式二： Plan-and-Execute想好再做

step1: 先规划，主agent先思考好完整的执行计划是什么 step2: 再执行，按制定的执行计划逐步完成，不用每步一思考；适合目标明确，稳定的任务

eg: 我现在要收拾行李准备出差

出差前物品清单和待办事项： 查天气，准备衣物和必需品，确认酒店，准备证件等

优点：省token，不需反复推理，只规划一次

缺点：不够灵活，计划有变（But, 可加入重新规划检查点，每隔几步检查一下计划是否靠谱）

模式三： Multi-Agent（团队协作）

适合场景：任务太复杂，一个agent搞不定，让多agent各司其职，适合任务天然可以拆成读个注意力状态的场景；典型框架：LangGraph(具体涉及流程框架小吴老师直播中讲过)

比如：一个规划，一个搜集信息，一个写代码，一个测试 在一个多agent场景中，每个agent内部可能用的都是ReAct模式

所以，我们在构写agent项目时，要提前考虑任务是否复杂，能够还分成多个子任务并行时，多agent才值得引入；如果不行，一个强大的agent足够。

工具调用与协议

Function Calling

让大模型发生了从只会说话到会做事的转变

工作模式

1、定义函数：目的，告诉LLM，开发者手里有哪些工具可以使用

用json格式描述每个函数名字、功能说明及参数

```
def get_weather(city: str) -> str:
    """查询城市天气"""
    # 实际项目中替换为真实天气 API
    weather_data = {
        "北京": "晴, 25°C, 湿度 40%",
        "上海": "多云, 28°C, 湿度 65%",
        "深圳": "阵雨, 30°C, 湿度 80%",
    }
    return weather_data.get(city, f"暂无 {city} 的天气数据")
```

```
{
  "type": "function",
  "function": {
    "name": "get_weather",
    "description": "查询指定城市的天气",
    "parameters": {
      "type": "object",
      "properties": {
        "city": {
          "type": "string",
          "description": "城市名, 如 北京、上海"
        }
      },
      "required": ["city"]
    }
  }
}
```

2、模型判断：目的，LLM要分析用户意图，回答这个问题，应该调用哪些函数

```
"function":{
  "name":"get_weather",
  "arguments":{"city":"上海"}
}
```

3、执行函数：注意，不是LLM执行函数，LLM只是告诉你该执行这个函数，具体是你程序代码拿到LLM解析出的调用指令，再去调api拿结果

4、生成回答：代码拿到真实数据后，将天气数据发给LLM，总结出答案

在一个agent中，包含多次的function call使用，调用完观察结果，决定下一次是否继续

MCP协议(agent<->tools)

若想agent强大，就要让LLM能连的上外部工具及服务；虽然function call也能解决，但不至于每种工具每个服务都写出适配代码，流程繁琐，参数结构差异大。

由此引出MCP(模型上下文协议) $\Rightarrow$ "USB-C"，工具接入层的标准化，让工具以统一方式暴露给 Agent

工作模式

- 1、MCP Host：一个agent应用，比如：IDE，聊天应用。
- 2、MCP Client：客户端，需要与MCP Server服务端进行通信；host请求什么，Client就与对应Server沟通。
- 3、MCP Server：服务端，对外暴露具体工具能力和资源。

用户query $\rightarrow$ Host——(Client寻找可用工具) $\rightarrow$ LLM(调用什么工具)——(Server接受调用请求，执行操作，返回结果) $\rightarrow$ LLM(基于结果生成回答)

A2A协议(agent $\leftrightarrow$ agent)

在一个多agent系统中，多agent之间的协作靠什么？怎么通信？

例：企业级多agent办公系统（主agent，数据查询agent，文档处理agent，通知推送agent等）

工作模式

- 1、Agent Card（智能体卡片）：要描述agent身份和具备什么能力，交互方式，认证信息等。
- 2、Task：主agent发配任务给子agent，任务要有完整的生命周期，涵盖从开始处理到结束的各个环节状态变化，方便追踪。
- 3、Message and Artifact：agent之间通信的渠道，agent完成任务后总结报告发给请求方。

插播一条八股面试基础问题：MCP和Function Calling、Skill区别？

回答模式：原理+区别

偏口语化回答示例：

"Function Calling解决调用层问题，模型表达出要调用A函数，参数有XXX，它更像是模型和应用之间的约定，工具来帮忙完成；MCP解决协议层问题。它并不是只管某一次的tool call，而是系统怎么暴露能力，怎么协商协议，怎么维持链接状态等事情进行统一，更像是标准化的连接层；Skill偏应用层封装，我的理解是，skill把一类任务能力打包成可复用的模块，里面可能包含prompt，工具链，状态逻辑，执行策略等，它关注的是能力复用，而不是协议本身；总结来说就是一个调用接口，一个连接协议，一个能力封装，这三个经常叠加使用"

Multi-Agent

把一个大任务拆给多个职责清晰的 agent 去做，它们之间通过某种方式通信和协作。

拆解任务很简单，真正困难的是：谁拥有决策权、子agent之间结果冲突怎么办、哪些状态共享、子agent卡住谁托管、结果如何合并、如何追踪agent的贡献和错误

Multi-Agent 的三种常见形态

1、父子型

主 agent 处理整个任务，遇到某个子问题时派一个 subagent 出去搞定，拿结果回来接着干。这是最常见的，Claude Code 里的 Task 工具就是这种

2、平级协作型

几个 agent 职责对等，通过共享状态或者消息互相协作。不过这种在工程上比较难落地，状态同步很麻烦。

3、主从型

有一个专门的协调者 agent，它自己不干活，只负责派 worker、收结果、做合成。worker 之间互不通信，全靠协调者调度。这种是高并发场景的标配。

Subagent 的隔离机制

为什么要隔离？

多 agent 系统本质就是一堆 agent 共处一个进程、共享一个底层运行时。如果隔离做得不好，一个 subagent 污染了父 agent 的状态、或者调了不该调的工具，整个系统就会乱成一锅粥。

隔离边界怎么设定？ (参考claude code中的subagent隔离机制)

1、工具隔离

为每个子agent分配独立的工具集合，标明哪些工具可调用，哪些不允许。不要假设所有agent都能用所有工具，按agent类型做细粒度权限控制。

- 设置通用黑名单(subagent不要干主agent的活)
 - 能派subagent的工具
 - 能主动问用户问题的工具
 - 能切换规划模式的工具
 - 能停止任务的工具
- 设置异步白名单(只通过安全的工具)
 - 读文件
 - 搜代码
 - 执行命令

2、上下文隔离

每一项状态单独判断该克隆、该共享、该屏蔽，还是该新建。

- 读文件缓存
 - 需复制给subagent独立的缓存副本，而不是共享同一个文件读取缓存
- 改全局状态
 - 限制subagent不能修改全局状态
- 保留subagent注册后台任务

- 让主agent从全局任务表中能找到subagent任务，不变成孤立进程
- 给subagent分发独立id
 - 主agent随时了解subagent嵌套在第几层

父子agent怎么通信

设想：子agent正在执行任务，父agent怎么发送新任务？subagent怎么传结果？

本质：消息队列+异步通知

- **父 → 子**：调 SendMessage 工具把消息写进子 agent 的信箱 → 子 agent 下一轮循环边界自己读取。
- **子 → 父**：子 agent 把完成通知拼成 XML 消息 → 伪装成用户消息（工具返回结果）注入父 agent 对话。

```
// src/tasks/LocalAgentTask/LocalAgentTask.tsx:116
export type LocalAgentTaskState = TaskStateBase & {
  type: 'local_agent';
  agentId: string;           // 子 agent 唯一 ID
  prompt: string;           // 初始任务
  agentType: string;
  status: TaskStatus;       // pending/running/completed/failed/killed
  result?: AgentToolResult; // 完成后的结果
  progress?: AgentProgress; // 进度
  isBackgrounded: boolean;  // 是否已转后台
  pendingMessages: string[]; // 信箱：父 agent 扔进来的待处理消息
  messages?: Message[];
};
```

整个通信体系没有直接函数调用，全靠读写共享的任务状态和消息队列，任务不依赖一次同步调用，可以后台运行，也可以恢复。

一次典型的通信流程：

- 1、父 Agent 创建任务，写入初始 prompt 和任务 ID。
- 2、子 Agent 读取任务，开始执行。
- 3、子 Agent 执行过程中更新 progress。
- 4、父 Agent 可以向 pendingMessages 写入补充指令。
- 5、子 Agent 下一轮观察时读取消息。
- 6、子 Agent 完成后写入 result。
- 7、父 Agent 读取 result，决定采纳、追问、重试或转交审核。

Multi-Agent 设计原则(总结)

- 上下文隔离要按字段粒度做（分清主和子，哪些该做，做什么，避免污染）
- 通信走消息，不走函数调用（异步，支持并发）
- 工具权限分级管控（黑名单，白名单）
- 缓存友好（复用已有信息，节约成本）
- 终止条件（防止占用资源，无限循环）
- 并行优先，协调合作

Agent系统落地最需要考虑的几个问题

上下文压缩

设想LLM收到信息包含：系统、历史对话提示词，工具输出内容，LLM回复，本次用户提示词

常见方法：

- 滑动窗口：保留近几轮对话，以往早期信息
- 压缩历史：压缩近百轮对话历史，保留关键信息
- 知识库RAG：历史记录向量化

针对于以上的方案，其实还有一些可优化的地方，可参考Claude code，在每次模型调用前**渐进式压缩**，重点可关注以下模块

注意！

压缩时最重要保留：

- 用户明确要求
- 已经做出的关键决策
- 当前任务进度
- 工具调用结果
- 失败原因
- 待办事项
- 不可改变的约束

- 裁剪
 - 删除空行，缩进代码块
 - 裁剪可缓存部分
 - 删除被覆盖的旧文本
- 微压缩
 - 标记重要部分为不可压缩片段
 - 不重要部分轻量压缩
 - 保留对话中的关键决策点和因果关系
- 上下文折叠
 - 完整对话历史压缩为精简摘要
 - 选轻量模型做摘要生成
 - 摘要进程和主进程并行
- 自动压缩
 - LLM摘要化整个对话历史
 - 用独立的子agent执行
 - 紧急强制终止对话窗口

记忆机制

- 用户画像：角色、偏好、知识水平
 - 记录用户是谁，擅长什么等特点信息
- 行为反馈：该做什么，不该做什么

- 不仅要记录规则，还要记录为什么，怎么做
- 项目动态：在做什么，截止日期，协作信息
 - 项目当前处理什么阶段，截止日期等
- 外部指针：哪能找到什么信息
 - 追踪项目进度

Agent项目面试问题

1、Memory 是 Agent 的一个关键模块。请问如何为 Agent 设计短期记忆和长期记忆系统？可以借助哪些外部工具或技术？

精简话术：短期记忆负责让 Agent 在当前任务中保持上下文连续，长期记忆负责让 Agent 跨会话积累用户偏好、业务知识和历史经验…。实际开发中通常会用 Redis 管短期状态，用数据库存结构化信息，用向量数据库加 RAG 管长期知识，再配合摘要、检索、权限、过期和更新机制，保证记忆既有用又可控。

分析：短期记忆主要解决的是“当前任务上下文”的问题，比如用户刚刚说了什么、当前工具调用结果是什么、任务执行到哪一步、下一步计划是什么。它通常存在于一次会话或一次任务链路中，生命周期较短；长期记忆解决的是“跨会话、跨任务的持续知识”问题。它存的是用户偏好、历史行为、业务知识、长期项目状态、已完成任务、常见问题、经验总结等。比如短期记忆，我们可以放在prompt context、session state、Redis、内存对象，或者数据库里的会话表中，就是让 Agent 在当前任务中保持连贯性，但是不能无限塞进上下文，通常需要做压缩，保留最近几轮对话和关键工具结果，对较长上下文做摘要，例如把几十轮对话压缩成当前任务状态；而长期记忆，我们可以借助RAG，Agent 在执行任务前，根据当前问题去长期记忆库中检索相关内容，再把高相关内容注入上下文。对于长期记忆，我们通常会把用户偏好，任务记忆，知识记忆，经验这些内容存放到milvus中

2、你的Agent如何处理tool call失败？有没有做过任何形式的容错设计？

精简话术：我的 Agent 处理 tool call 失败，没有设计成 try-catch，而用了调用前参数校验、调用中超时重试、调用后结果校验，再结合错误分类、幂等控制、降级策略和**日志追踪**。查询类工具可以自动重试，写操作要谨慎处理，必要时走人工兜底。核心目标是让 Agent 在工具失败时可恢复、可解释、可追踪，同时不能编造结果。

分析：首先调用失败有以下几种情况：1、参数校验失败，对于这类一般是模型生成的参数格式不太对、缺字段等因素，可以先做schema validation，所有工具都定义清晰的json schema或类型约束，调用工具前检查拦截，自动修正让模型重新生成参数；2、临时错误，网络超时，这类可以添加重试机制，需要设置最大重试次数；3、业务错误，需要把错误结构化返回给agent，让它界定下一步怎么做，紧急人工兜底；4、工具返回结果异常，需要对结果校验，或二次查询等，所以需要没tool call都要记录 trace_id、tool_name、参数摘要、耗时、错误码、重试次数、最终结果等

Agent项目简历如何写

1、不要堆技术名词

×写了LangGraph、LlamaIndex、ReAct这些框架名。

重点是讲清楚你让模型在复杂场景下，能自己判断该用哪个工具、怎么用、什么时候停。

2、有量化数据

工具调用率增长8%，压缩率16%等，数字比形容词有说服力

3、体现设计与思考

不要只写做了什么，要写为什么这么做。比如为什么选择分层缓存而不是全局缓存？为什么用checkpoint而不是重放整个对话？

4、覆盖完整链路

包括工具定义、控制策略、决策逻辑，监控反馈等维度，每个模块精简概括

模板参考

企业级多 Agent 智能工单处理与知识问答平台

项目背景： 公司内部客服、运维、售后知识分散在文档、历史工单、数据库和业务系统中，人工处理工单存在响应慢、重复问题多、知识检索效率低、答案不一致等问题。项目目标是构建一个基于 LLM + RAG + 多 Agent 协作的智能工单处理平台，实现问题理解、知识检索、工具调用、风险校验、自动回复和人工升级闭环。

技术栈： Python、FastAPI、LangGraph、LangChain、Qwen、Milvus、Redis、Docker

项目流程：

- 设计 **Coordinator Agent** 作为统一调度中心，基于用户意图、置信度、任务复杂度和风险等级动态选择执行路径，支持普通问答、数据查询、故障诊断、人工升级等多种处理模式。
- 构建 **Intent Agent** 对用户问题进行意图识别、领域分类和优先级判断，结合规则特征与 LLM 分类结果进行融合决策，将工单准确路由至对应业务 Agent，意图识别准确率达到 **91%+**。
- 实现 **RAG Retrieval Agent**，对企业文档、历史工单、FAQ 进行切片、向量化和混合检索，采用 **BM25 + 向量检索 + rerank** 策略提升召回质量，知识命中率由 **68% 提升至 86%**。
- 设计 **Tool Agent** 负责调用 CRM、订单、日志、监控等内部系统接口，通过 Function Calling 封装工具权限、参数校验和失败重试机制，使 Agent 能够完成订单查询、故障定位、状态核验等动作。
- 引入 **Risk & Guardrail Agent** 对回复内容进行事实一致性、敏感信息、权限边界和高风险操作检查，低置信度或高风险场景自动转人工，幻觉率由 **8.5% 降至 2.1%**。
- 建立 **Feedback Agent**，采集用户评分、人工改写、命中知识源、转人工原因等反馈数据，用于优化 prompt、检索策略和知识库更新，实现系统持续迭代。

项目上线后，工单平均首次响应时间从 **12 分钟降低至 35 秒**，自动解决率达到 **42%**，人工客服日均处理效率提升约 **55%**，LLM 调用成本通过缓存和模型分层策略降低约 **30%**。

Agent项目文档

.....